

Web Application Security Monitoring using Nginx, ModSecurity (WAF), and Wazuh SIEM

Project Overview

Modern web applications are constantly exposed to attacks such as Cross-Site Scripting (XSS), Local File Inclusion (LFI), directory traversal, and protocol abuse. Traditional perimeter security is not sufficient to detect and analyze these attacks at the application layer.

In this project, **Nginx** is deployed as a web server, **ModSecurity v3** is integrated as a **Web Application Firewall (WAF)** using the **OWASP Core Rule Set (CRS)**, and **Wazuh** is used as a **centralized security monitoring and SIEM platform**.

The objective is to:

- Protect a web application at Layer 7 using ModSecurity
- Detect real-world web attacks (XSS, LFI, common web attacks)
- Collect and analyze ModSecurity and Nginx logs using Wazuh
- Visualize and classify attacks inside the Wazuh dashboard with rule IDs, severity, MITRE mapping, and attack descriptions

This setup simulates a **real enterprise web security monitoring architecture**.



Tools & Technologies Used

- **Operating System:** Ubuntu Linux
- **Web Server:** Nginx 1.24
- **Web Application Firewall:** ModSecurity v3
- **Rule Engine:** OWASP Core Rule Set (CRS 3.x)
- **SIEM / Log Monitoring:** Wazuh Manager & Wazuh Agent
- **Log Sources:**
 - Nginx access.log
 - Nginx error.log

- ModSecurity audit and error logs
- **Testing Tools:**
 - curl (manual attack simulation)

Why ModSecurity + Wazuh?

Why ModSecurity?

- Acts as a **Web Application Firewall (WAF)**
- Detects and blocks application-layer attacks
- Uses OWASP CRS rules maintained by the security community
- Protects against OWASP Top 10 vulnerabilities

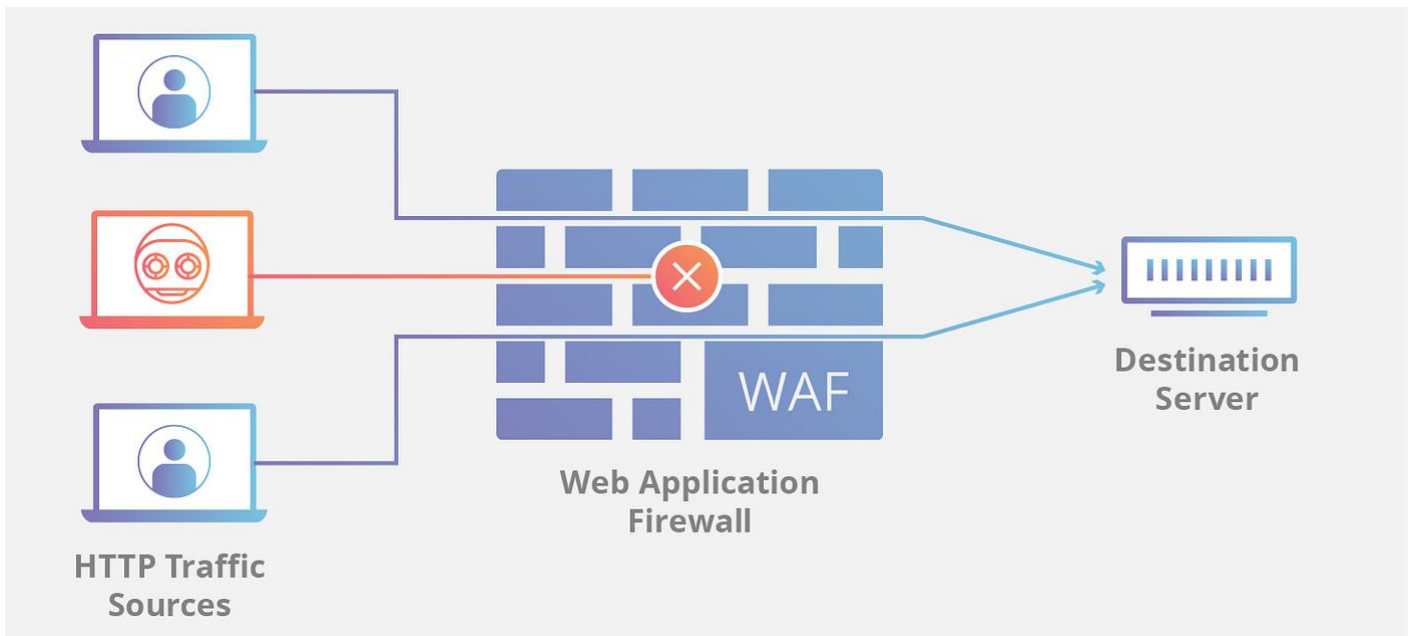
Why Wazuh Integration?

- Centralized monitoring and alerting
- Correlates web attacks with MITRE ATT&CK
- Provides dashboards, severity levels, and rule descriptions
- Enables SOC-style visibility for web application security

Together, ModSecurity blocks attacks while **Wazuh provides visibility, detection, and investigation capabilities.**

Architecture Overview

```
Attacker
|
v
[Nginx Web Server]
|
[ModSecurity (WAF) + OWASP CRS]
|
[Log Files: access.log / error.log]
|
[Wazuh Agent]
|
[Wazuh Manager]
|
[Wazuh Dashboard]
```



Implementation Steps

Step 1: Install Nginx Web Server

Purpose: Deploy a web server to host the application and act as the inspection point for ModSecurity.

- Nginx service is installed and enabled
- Service status verified using systemctl

Result: Nginx service is running successfully

```
root@ip-10-10-10-10: /home/ubuntu# sudo systemctl enable nginx
sudo systemctl start nginx
Synchronizing state of nginx.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable nginx
root@ip-10-10-10-10: /home/ubuntu# systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-02-01 23:03:30 IST; 14s ago
     Docs: man:nginx(8)
  Main PID: 9348 (nginx)
    Tasks: 5 (limit: 19161)
   Memory: 3.7M (peak: 8.0M)
      CPU: 30ms
   CGroup: /system.slice/nginx.service
           └─9348 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─9350 "nginx: worker process"
               └─9351 "nginx: worker process"
                 └─9352 "nginx: worker process"
                   └─9353 "nginx: worker process"

Feb 01 23:03:30 ip-10-10-10-10 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Feb 01 23:03:30 ip-10-10-10-10 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
root@ip-10-10-10-10: /home/ubuntu# curl -I http://10.10.10.10
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Sun, 01 Feb 2026 17:34:19 GMT
Content-Type: text/html
Content-Length: 615
Last-Modified: Sun, 01 Feb 2026 17:33:30 GMT
Connection: keep-alive
ETag: "697f8e6a-267"
Accept-Ranges: bytes
```

Step 2: Verify Nginx in Browser

Purpose: Confirm that the web server is accessible before adding security controls.

- Accessed the server IP in a browser

- Default Nginx welcome page is displayed

Result: Web server is reachable and working correctly



Step 3: Install ModSecurity v3 (Library Mode)

Purpose: Install ModSecurity as a standalone security engine compatible with Nginx.

- ModSecurity source code cloned
- Build and configuration process initiated

Result: ModSecurity core library prepared for integration with Nginx

```
root@ip-10-10-10-10:~# cd /home/ubuntu# cd /opt
sudo git clone https://github.com/SpiderLabs/ModSecurity
cd ModSecurity
sudo git submodule init
sudo git submodule update
Cloning into 'ModSecurity'...
remote: Enumerating objects: 44421, done.
remote: Counting objects: 100% (581/581), done.
remote: Compressing objects: 100% (171/171), done.
remote: Total 44421 (delta 456), reused 410 (delta 410), pack-reused 43840 (from 2)
Receiving objects: 100% (44421/44421), 75.94 MiB | 31.33 MiB/s, done.
Resolving deltas: 100% (33595/33595), done.
Submodule 'bindings/python' (https://github.com/owasp-modsecurity/ModSecurity-Python-bindings.git) registered for path 'bindings/python'
Submodule 'others/libinjection' (https://github.com/libinjection/libinjection.git) registered for path 'others/libinjection'
Submodule 'others/mbedtls' (https://github.com/Mbed-TLS/mbedtls.git) registered for path 'others/mbedtls'
Submodule 'test/test-cases/seccules-language-tests' (https://github.com/owasp-modsecurity/seccules-language-tests) registered for path 'test/test-cases/seccules-language-tests'
Cloning into '/opt/ModSecurity/bindings/python'...
Cloning into '/opt/ModSecurity/others/libinjection'...
```

Step 4: Install Build Dependencies and Essentials

Purpose: Install required development libraries to compile ModSecurity and Nginx modules.

Installed packages include:

- gcc, make, build-essential
- libpcre, libxml2, libssl
- libyajl, libltdb, curl

Result: System is ready for compilation

```
root@ip-10-10-10-10:~# sudo apt update
sudo apt install -y \
git gcc make build-essential \
libpcre3 libpcre3-dev \
libssl-dev libxml2 libxml2-dev \
libyajl-dev libgeoip-dev \
libltdb-dev libtool autoconf \
pkg-config curl
```

Step 5: Build ModSecurity from Source

Purpose: Compile ModSecurity v3 library.

- build.sh executed
- configure, make, and make install completed

Result: libmodsecurity installed under /usr/local/modsecurity

```
root@ip-10-10-10-10: /opt/ModSecurity# sudo ./build.sh
sudo ./configure
sudo make
sudo make install
libtoolize: putting auxiliary files in '.'.
libtoolize: copying file './ltmain.sh'
libtoolize: putting macros in AC_CONFIG_MACRO_DIRS, 'build'.
libtoolize: copying file 'build/libtool.m4'
libtoolize: copying file 'build/ltoptions.m4'
libtoolize: copying file 'build/ltsugar.m4'
libtoolize: copying file 'build/ltversion.m4'
libtoolize: copying file 'build/lt~obsolete.m4'
```

Step 6: Verify ModSecurity Installation

Purpose: Ensure ModSecurity library is correctly installed and recognized.

- ldconfig updated
- pkg-config used to verify ModSecurity version

Result: ModSecurity v3.0.14 successfully installed

```
mv -f optimization/.deps/rules_optimization-optimization.Tpo optimization/.deps/rules_optimization-optimization.Po
/bin/bash ../libtool --tag=CXX --mode=link g++ -std=c++17 -g -O2 -L../src/.libs/ -lmodsecurity -lpthread -lm -lstdc++ -o r
ization/rules_optimization-optimization.o -lcurl -lGeoIP -lrt -lxml2 -llmdb -lpcres2-8 -lyajl
libtool: link: g++ -std=c++17 -g -O2 -o .libs/rules_optimization optimization/rules_optimization-optimization.o -L../src/.libs/ /opt
libmodsecurity.so -lpthread -lm -lstdc++ -lcurl -lGeoIP -lrt -lxml2 -llmdb -lpcres2-8 -lyajl -Wl,-rpath -Wl,/usr/local/modsecurity/lib
make[3]: Entering directory '/opt/ModSecurity/test'
make[3]: Nothing to be done for 'install-data-am'.
make[3]: Leaving directory '/opt/ModSecurity/test'
make[2]: Leaving directory '/opt/ModSecurity/test'
make[1]: Leaving directory '/opt/ModSecurity/test'
make[1]: Entering directory '/opt/ModSecurity'
make[2]: Entering directory '/opt/ModSecurity'
make[2]: Nothing to be done for 'install-exec-am'.
  /usr/bin/mkdir -p '/usr/local/modsecurity/lib/pkgconfig'
  /usr/bin/install -c -m 644 modsecurity.pc '/usr/local/modsecurity/lib/pkgconfig'
make[2]: Leaving directory '/opt/ModSecurity'
make[1]: Leaving directory '/opt/ModSecurity'
```

Step 7: Clone and Build ModSecurity–Nginx Connector

Purpose: Enable ModSecurity support inside Nginx using a dynamic module.

- ModSecurity-nginx repository cloned
- Nginx source compiled with ModSecurity module

Result: ngx_http_modsecurity_module.so generated

```

root@ip-10-10-10-10:/opt/ModSecurity# echo "/usr/local/modsecurity/lib" | sudo tee /etc/ld.so.conf.d/modsecurity.conf
sudo ldconfig
/usr/local/modsecurity/lib
root@ip-10-10-10-10:/opt/ModSecurity# ldconfig -p | grep modsecurity
libmodsecurity.so.3 (libc6,x86-64) => /usr/local/modsecurity/lib/libmodsecurity.so.3
libmodsecurity.so (libc6,x86-64) => /usr/local/modsecurity/lib/libmodsecurity.so
root@ip-10-10-10-10:/opt/ModSecurity# cd /opt
sudo git clone https://github.com/SpiderLabs/ModSecurity-nginx.git
Cloning into 'ModSecurity-nginx'...
remote: Enumerating objects: 1160, done.
remote: Counting objects: 100% (328/328), done.
remote: Compressing objects: 100% (68/68), done.
remote: Total 1160 (delta 284), reused 265 (delta 260), pack-reused 832 (from 3)
Receiving objects: 100% (1160/1160), 1.39 MiB | 17.77 MiB/s, done.
Resolving deltas: 100% (768/768), done.
root@ip-10-10-10-10:/opt#

```

Step 8: Troubleshooting and Path Fixes

Purpose: Resolve module path mismatches and missing dependencies.

- Corrected Nginx module directory
- Fixed library paths and unicode.mapping issues

Result: Nginx recognizes ModSecurity module

```

cc -c -fPIC -pipe -O -W -Wall -Wpointer-arith -Wno-unused-parameter -Werror -g -I /usr/local/modsecurity/include -I objs -I src/http -I src/http/modules \
    -o objs/nginx_http_modsecurity_module_modules.o \
    objs/nginx_http_modsecurity_module_modules.c
cc -o objs/nginx_http_modsecurity_module.so \
    objs/addon/src/nginx_http_modsecurity_module.o \
    objs/addon/src/nginx_http_modsecurity_access.o \
    objs/addon/src/nginx_http_modsecurity_header_filter.o \
    objs/addon/src/nginx_http_modsecurity_body_filter.o \
    objs/addon/src/nginx_http_modsecurity_log.o \
    objs/nginx_http_modsecurity_module_modules.o \
    -Wl,-rpath,/usr/local/modsecurity/lib -L/usr/local/modsecurity/lib -lmodsecurity \
    -shared
make[1]: Leaving directory '/opt/nginx-1.24.0'
root@ip-10-10-10-10:/opt/nginx-1.24.0# ls objs/nginx_http_modsecurity_module.so
objs/nginx_http_modsecurity_module.so

```

Step 9: Final Configuration and Nginx Validation

Purpose: Enable ModSecurity and load OWASP CRS rules.

- modsecurity on enabled in Nginx config
- OWASP CRS linked and configured
- nginx -t executed successfully

Result: Nginx starts with ModSecurity enabled


```

root@ip-10.0.1.20:/opt/nginx-1.24.0# sudo cp /usr/share/modsecurity-crs/crs-setup.conf.example \
/etc/nginx/modsec/crs/crs-setup.conf
cp: cannot stat '/usr/share/modsecurity-crs/crs-setup.conf.example': No such file or directory
root@ip-10.0.1.20:/opt/nginx-1.24.0# ls /usr/share/modsecurity-crs/
owasp-crs.load  rules  util
root@ip-10.0.1.20:/opt/nginx-1.24.0# sudo cp /usr/share/modsecurity-crs/owasp-crs.load \
/etc/nginx/modsec/crs/owasp-crs.load
cp: '/usr/share/modsecurity-crs/owasp-crs.load' and '/etc/nginx/modsec/crs/owasp-crs.load' are the same file
root@ip-10.0.1.20:/opt/nginx-1.24.0# sudo nano /etc/nginx/modsec/crs/crs-setup.conf
root@ip-10.0.1.20:/opt/nginx-1.24.0# sudo nano /etc/nginx/modsec/crs/crs-setup.conf
root@ip-10.0.1.20:/opt/nginx-1.24.0# sudo nano /etc/nginx/modsec/crs/owasp-crs.load
root@ip-10.0.1.20:/opt/nginx-1.24.0# sudo nano /etc/nginx/modsec/main.conf
root@ip-10.0.1.20:/opt/nginx-1.24.0# ls /etc/nginx/modsec/modsecurity.conf
/etc/nginx/modsec/modsecurity.conf
root@ip-10.0.1.20:/opt/nginx-1.24.0# sudo nano /etc/nginx/modsec/modsecurity.conf
root@ip-10.0.1.20:/opt/nginx-1.24.0# sudo nginx -t
2026/02/01 23:53:03 [notice] 110110#110110: ModSecurity-nginx v1.0.4 (rules loaded inline/local/remote: 0/920/0)
2026/02/01 23:53:03 [notice] 110110#110110: libmodsecurity3 version 3.0.14
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
root@ip-10.0.1.20:/opt/nginx-1.24.0#

```

Step 10: Verify ModSecurity Protection

Purpose: Test whether ModSecurity blocks malicious requests.

Tests performed:

- XSS payload
- Local File Inclusion (LFI)

Result: Requests blocked with HTTP 403 response

```

root@ip-10.0.1.20:/opt/nginx-1.24.0# curl "http://3.217.85.2/?q=<script>alert(1)</script>"
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.24.0 (Ubuntu)</center>
</body>
</html>
root@ip-10.0.1.20:/opt/nginx-1.24.0# curl "http://3.217.85.2/?file=../../etc/passwd"
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.24.0 (Ubuntu)</center>
</body>
</html>
root@ip-10.0.1.20:/opt/nginx-1.24.0# tail -f /var/log/nginx/error.log

```

Step 11: Analyze Raw ModSecurity and Nginx Logs

Purpose: Validate detection at log level.

- error.log shows ModSecurity rule matches
- CRS rule IDs, severity, and attack types visible

Result: Web attacks clearly logged and identified

```

2026/02/02 00:00:36 [error] 119892#119892: *3 [client 2.217.25.2] ModSecurity: Access denied with code 403 (phase 2). Matched "Operator 'Rx' with parameter
'^[\\d.:]+$', against variable 'REQUEST_HEADERS:Host' (Value: '2.217.25.2') [file "/etc/nginx/modsec/crs/rules/REQUEST-920-PROTOCOL-ENFORCEMENT.conf"] [line
"719"] [id "920350"] [rev "" ] [msg "Host header is a numeric IP address"] [data "2.217.25.2"] [severity "4"] [ver "OWASP CRS/3.3.5"] [maturity "0"] [accurac
y "0"] [tag "application-multi"] [tag "language-multi"] [tag "platform-multi"] [tag "attack-protocol"] [tag "paranoia-level/1"] [tag "OWASP_CRS"] [tag "cape
c/1000/210/272"] [tag "PCI/6.5.10"] [hostname ""] [uri "/" ] [unique_id "176997063691.162777"] [ref "o0,10v49,10"], client: 2.217.25.2, server: _,
request: "GET /?q=<script>alert(1)</script> HTTP/1.1", host: "
2026/02/02 00:00:43 [error] 119893#119893: *4 [client 2.217.25.2] ModSecurity: Access denied with code 403 (phase 2). Matched "Operator 'Rx' with parameter
'^[\\d.:]+$', against variable 'REQUEST_HEADERS:Host' (Value: '2.217.25.2') [file "/etc/nginx/modsec/crs/rules/REQUEST-920-PROTOCOL-ENFORCEMENT.conf"] [line
"719"] [id "920350"] [rev "" ] [msg "Host header is a numeric IP address"] [data "2.217.25.2"] [severity "4"] [ver "OWASP CRS/3.3.5"] [maturity "0"] [accurac
y "0"] [tag "application-multi"] [tag "language-multi"] [tag "platform-multi"] [tag "attack-protocol"] [tag "paranoia-level/1"] [tag "OWASP_CRS"] [tag "cape
c/1000/210/272"] [tag "PCI/6.5.10"] [hostname ""] [uri "/" ] [unique_id "176997064372.694119"] [ref "o0,10v43,10"], client: 2.217.25.2, server: _,
request: "GET /?file=../../etc/passwd HTTP/1.1", host: "

```

Step 12: Integrate Nginx & ModSecurity Logs with Wazuh

Purpose: Enable centralized monitoring.

- Wazuh agent configured to monitor:
 - /var/log/nginx/access.log
 - /var/log/nginx/error.log

Result: Logs successfully forwarded to Wazuh manager

```

<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/nginx/error.log</location>
</localfile>

<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/nginx/access.log</location>
</localfile>

```

Step 13: Simulate Web Attacks

Purpose: Generate real attack events.

Attacks tested:

- XSS attack using script injection
- File traversal and LFI attempts

Result: Attacks detected and blocked

```

root@ip-10.0.1.20: /opt/nginx-1.24.0# curl "http://2.217.25.2/?q=<script>alert(1)</script>"
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.24.0 (Ubuntu)</center>
</body>
</html>
root@ip-10.0.1.20: /opt/nginx-1.24.0# curl "http://2.217.25.2/?file=../../etc/passwd"
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.24.0 (Ubuntu)</center>
</body>
</html>
root@ip-10.0.1.20: /opt/nginx-1.24.0# tail -f /var/log/nginx/error.log

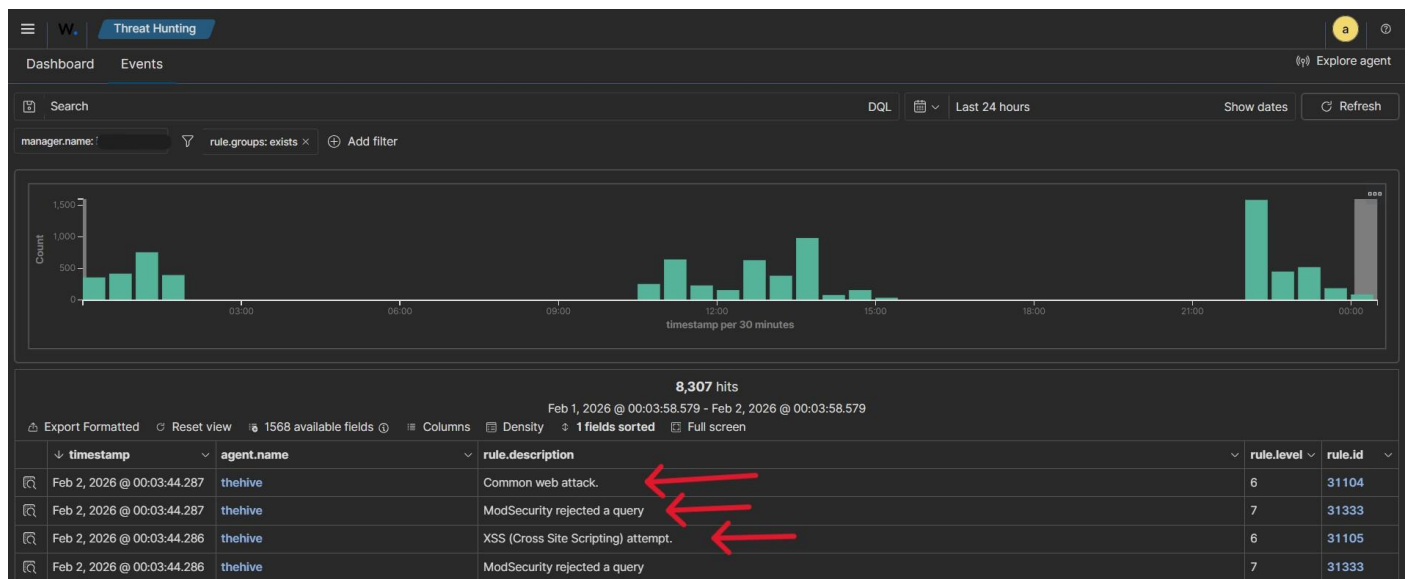
```


Step 14: Verify Alerts in Wazuh Dashboard

Purpose: Confirm detection and classification.

Observed in Wazuh:

- ModSecurity rejected query
- XSS (Cross Site Scripting) attempt
- Common web attack detection Each alert includes:
- Rule ID
- Severity level
- Attack category
- MITRE ATT&CK mapping



t agent.name	thehive
t data.srcip	[REDACTED]
t decoder.name	nginx-errorlog
t decoder.parent	nginx-errorlog
t full_log	2026/02/02 00:03:43 [error] 119890#119890: *5 [client [REDACTED]] ModSecurity: Access denied with code 403 (phase 2). Matched "Operator `Rx' with parameter `^[\d.:]+\$' against variable `REQUEST_HEADERS:Host' (Value: `[REDACTED]') [file "/etc/nginx/modsec/crs/rules/REQUEST-920-PROTOCOL-ENFORCEMENT.conf"] [line "719"] [id "920350"] [rev ""] [msg "Host header is a numeric IP address"] [data [REDACTED]] [severity "4"] [ver "OWASP_CRS/3.3.5"] [maturity "0"] [accuracy "0"] [tag "application-multi"] [tag "language-multi"] [tag "platform-multi"] [tag "severity-4"] [tag "wasc-9"]
t id	1769970824.4596650
t input.type	log
t location	/var/log/nginx/error.log
t manager.name	i, [REDACTED]
t rule.description	ModSecurity rejected a query
# rule.firedtimes	1
t rule.gpg13	10.1
t rule.groups	nginx, web, modsecurity
t rule.id	31333
# rule.level	7
🔊 rule.mail	false
t rule.mitre.id	T1083
t rule.mitre.tactic	Discovery
t rule.mitre.technique	File and Directory Discovery
📅 timestamp	Feb 2, 2026 @ 00:03:44.286

Wazuh Alert Analysis

XSS Attack Alert

- Description: XSS (Cross Site Scripting) attempt
- Protocol: HTTP GET
- Decoder: web-accesslog
- Rule ID: 31105
- Severity: Medium–High
- MITRE Technique: JavaScript Execution

t data.id	403
t data.protocol	GET
t data.srcip	[REDACTED]
t data.url	/?q=<script>alert(1)</script>;
t decoder.name	web-accesslog
t full_log	2 - - [02/Feb/2026:00:03:43 +0530] "GET /?q=<script>alert(1)</script> HTTP/1.1" 403 162 "-" "curl/8.5.0"
t id	1769970824.4596179
t input.type	log
t location	/var/log/nginx/access.log
t manager.name	[REDACTED]
t rule.description	XSS (Cross Site Scripting) attempt.
# rule.firedtimes	1
t rule.gdpr	IV_35.7.d
t rule.groups	web, accesslog, attack
t rule.id	31105
# rule.level	6
🔍 rule.mail	false
t rule.mitre.id	T1059.007
t rule.mitre.tactic	Execution
t rule.mitre.technique	JavaScript
t rule.nist_800_53	SA.11, SI.4
t rule.pci_dss	6.5, 11.4, 6.5.7
t rule.tsc	CC6.6, CC7.1, CC8.1, CC6.1, CC6.8, CC7.2, CC7.3
📅 timestamp	Feb 2, 2026 @ 00:03:44.286

File Traversal / LFI Alert

- Description: Common web attack
- URL: ../../etc/passwd
- Rule Groups: web, accesslog, attack
- MITRE Techniques:
 - File and Directory Discovery
 - Initial Access

t agent.name	thehive
t data.id	403
t data.protocol	GET
t data.srcip	
t data.url	<u>/?file=../../etc/passwd</u>
t decoder.name	<u>web-accesslog</u>
t full_log	3.217.85.2 - - [02/Feb/2026:00:03:43 +0530] "GET /?file=../../etc/passwd HTTP/1.1" 403 162 "-" "curl/8.5.0"
t id	1769970824.4597706
t input.type	log
t location	/var/log/nginx/access.log
t manager.name	
t rule.description	<u>Common web attack.</u>
# rule.firedtimes	1
t rule.gdpr	IV_35.7.d
t rule.groups	<u>web, accesslog, attack</u>
t rule.id	31104
# rule.level	6
rule.mail	false
t rule.mitre.id	T1055 T1083 T1190
t rule.mitre.tactic	<u>Defense Evasion, Privilege Escalation, Discovery, Initial Access</u>
t rule.mitre.technique	<u>Process Injection, File and Directory Discovery, Exploit Public-Facing Application</u>
t rule.nist_800_53	SA.11, SI.4
t rule.pci_dss	6.5, 11.4, 6.5.1
t rule.tsc	CC6.6, CC7.1, CC8.1, CC6.1, CC6.8, CC7.2, CC7.3
timestamp	Feb 2, 2026 @ 00:03:44.287

Project Outcome

- Successfully deployed Nginx with ModSecurity WAF
- OWASP CRS actively blocking web attacks
- Wazuh providing full visibility and alerting
- Real-world web attack detection validated

Skills Gained

- Web Application Firewall deployment
- ModSecurity v3 compilation and troubleshooting

- OWASP CRS configuration
- Nginx dynamic module handling
- SIEM integration using Wazuh
- Web attack detection and analysis
- Log-based security monitoring

Conclusion

This project demonstrates a **complete end-to-end web application security monitoring solution**. By combining **Nginx, ModSecurity, and Wazuh**, application-layer attacks are not only blocked but also **detected, logged, analyzed, and visualized** in a SIEM platform.

This setup closely mirrors **enterprise SOC and WAF deployments** and provides a strong foundation for advanced security monitoring, incident response, and threat hunting in web environments.
